Rendszeraudit — "Lyukak a pajzson"

Teljes átvilágítás a production és a TDK-konferencia előtt · 2026. szeptember

A teljes projekt (TDK-dolgozat, README-k, backend, frontend, Docker, mérőszkriptek, böngésző-kiegészítő) átvilágítása a production és a konferencia előtt. Minden megállapítás a kódból, futtatott ellenőrzésekből vagy a dolgozat szövegéből származik.

1. Összegzés

Kategória	Darab	Állapot
Kritikus, éles hiba	6	Javítva (lásd 02-es dokumentum)
Magas prioritású hiba	11	Nagyrészt javítva
Közepes prioritású	10	Részben javítva
Holt kód / felesleg	7 csoport	Eltávolítva

---

2. A dolgozat és a valós kód szétvált

A 4–5. fejezet egy **webhook + Express.js + Socket.io relay** architektúrát ír le (`/subscribe_webhook`, külön Node szerver). A ténylegesen futó kód ennél más és kifinomultabb: natív FastAPI WebSocket a `/ws` végponton, kliensenkénti állapotgéppel (gesztus-puffer, tartási idő, lecsengési idő, „parancs mód" ablak).

Teendő a konferencia előtt: a dolgozat architektúra-leírását a ténylegesen futó WebSocket-alapú megoldásra kell frissíteni. A régi webhook-leírás ma már nem létező kódot mutat be.

A dolgozatban közölt mérési adatok (22/8 és 20/10, $\chi^2 = 0{,}079$) és a repóban lévő mérőszkript adatai (30/0, 28/2) eltérnek. A szerző jelezte, hogy azóta újabb mérések készültek — ezeket a végleges dolgozatban egységesíteni kell.

3. Megerősített, reprodukált éles hibák

A dokumentált helyi indítási parancs nem működött. A `uvicorn app.main:app` a repó gyökeréből `ModuleNotFoundError: No module named 'api'` hibával állt le, mert a belső importok „lapos" formát használtak, ami csak a konténerben (`PYTHONPATH=/app`) oldódott fel.

Hardkódolt, csak konténerben létező modell-útvonal. A `model_asset_path="/app/hand_landmarker.task"` abszolút útvonal miatt a helyi futtatás `RuntimeError: Unable to open file` hibával elszállt.

A CRA proxy Docker-belső hosztnevet hardkódolt (`"proxy": "http://api:8000"`), ami bare-metal `npm start` esetén nem oldható fel. Ez volt a legvalószínűbb oka annak, hogy a WebSocket-kapcsolat nem működött.

A fejlesztői `docker-compose.yml`-ből hiányzott a reverse proxy. A frontend konténer nyers dev szerverként futott, így a same-origin `/ws` és `/api` hívásoknak nem volt hová menniük.

Megosztott, mutable gesztusfelismerő állapot. Egyetlen, modulszintű detektor példányt használt minden kliens, miközben a detektor példányszintű állapotot tartott a swipe-sebesség

számításához. Két egyidejű felhasználó esetén az egyik kéz mozgása felülírta a másik bázisértékét.

A testreszabott gesztus→funkció leképezés csak angolul működött. A beállítások a lefordított gesztusnévvel voltak kulcsolva, a kiolvasás viszont a nyers backend-literállal történt.

4. Holt kód

Fájl	Miért halott
`app/fps.py`	209 soros prototípus, elavult `mp.solutions` API-val; semmi nem importálta
`app/models/pose_detectation.py`	Sosem példányosítva; a `process_pose()` no-op stub volt
`app/api/process.py`	Egyetlen globális detektorpéldányt tartott — kiváltva a session-réteggel
`app/client/*.py`	Három teszt-kliens a törölt `/subscribe_webhook` végpontokra
`app/db.py` kommentelt hívások	Minden adatbázis-írás ki volt kommentezve
Frontend csomagok	`socket.io`, `socket.io-client`, `cors`, `axios`, `swr`, `tsparticles`, `react-tsparticles`, `lucide-react`, `@mediapipe/hands`, `@mediapipe/drawing_utils`
Frontend fájlok	`background.jpg`, `vback.jpg`, `logo.svg`, `App.test.js`

Megtartva a fejlesztő kérésére: `app/models/segmentation.py`, `upsert_client` (bekötve), `browser_extension/`.

5. Teljesítmény és MediaPipe

A saját mérési táblázat (6.1. táblázat) egyértelmű: a MediaPipe 15–20 FPS-t hoz GPU nélkül, míg a Meta Sapiens 0,002–0,02, a YOLOv8 2–7 FPS-t. **A MediaPipe a helyes választás** — a nyerhető sebesség a használat módjában volt.

A HandLandmarker `IMAGE` **módban futott** `VIDEO` helyett, ami minden képkockán újrafuttatta a teljes tenyérdetektálást a beépített követés helyett. Ez a legvalószínűbb oka a mért 276 ms átlag mögötti 188,7 ms szórásnak és a 828 ms-os csúcsnak.

Két eltérő képküldési mechanizmus működött párhuzamosan: a Presentation oldal 100 ms-onként teljes felbontású JPEG-et küldött HTTP POST-tal (minden ciklusban új canvas elemet létrehozva, visszanyomás-kezelés nélkül), a Practice oldal 180 ms-onként 640 px-es képet WebSocketen.

A MediaPipe wasm/modell fájlok publikus CDN-ről töltődtek, ami offline vagy korlátozott hálózaton hibaforrás.

6. Biztonság

- CORS `allow_origins=["*"]` és `allow_credentials=True` együtt — a CORS-specifikáció szerint érvénytelen kombináció.

- Nem volt hitelesítés: a `clientId` kliens által generált, kitalálható string volt, amivel bárki rácsatlakozhatott más gesztusfolyamára.
- Nem volt rate limit és bemenet-validáció a képfogadó végponton.
- Titkosítatlan adatbázis-jelszó a verziókezelt Compose fájlokban.
- Egyetlen függőség sem volt verzióhoz rögzítve.

7. Adatbázis és adatkezelés

A Postgres réteg futott, migrált, de **semmit nem írt bele senki** — minden hívási hely ki volt kommentezve. A hozzájárulás (consent) rögzítése így csak látszat volt: a frontend elküldte, a backend eldobta.

Pozitív, megőrzendő tulajdonság: **a nyers képkockák soha nem kerülnek tárolásra**, csak memóriában dolgozódnak fel. Ez valódi privacy-by-design, amit a dokumentációban is rögzíteni kell.

8. Kutatási és mérési hiányosságok

Jelenlegi állapot	Probléma	Mi kell helyette
n=1 személy, n=30 próba, 2 gesztus	Nincs statisztikai erő, nincs interperszonális variancia	Több résztvevő, minden gesztus, előre rögzített protokoll
Konfúziós mátrix egyetlen valódi osztályra	Nem valódi többosztályos mátrix	Minden osztályt lefedő mátrix
Kliens és API ugyanazon a gépen	Nulla valódi hálózati késleltetés	Mérés valós hálózat felett
Egyetlen aggregált latencia-szám	Nem tudni, hol a nyerhető idő	Komponensekre bontott latencia
Kézzel bemásolt adattömbök a szkriptekben	Nem reprodukálható	Élő adatforrásból számoló szkriptek
Nincs beépített telemetria	A rendszer nem gyűjt adatot magáról	Adatbázisba írt esemény- és latencia-napló
Nincs többklienses terhelési teszt	A skálázhatósági állítás alátámasztatlan	Konkurrens kliens-szám vs. latencia mérés

9. UX kód szinten

- Nem volt WebSocket `onerror`/`onclose` kezelés: megszakadt kapcsolatnál a felület némán befagyott.
- A Practice oldal olyan gesztusokat kínált gyakorlásra, amiket a backend nem tudott visszaigazolni.
- A lézermutató koordináta-ága halott kód volt: a backend sosem küldött koordinátát.
- A kamera-engedély hibája nem jutott el a felhasználóhoz.